Exact k-Dispersion for Points in Convex Position: a Capped Delaunay Dynamic Program, Search Windows, and the Limits of Greedy Structure

Project Team

October 27, 2026

Abstract

Given n points in convex position in the plane and an integer k, the k-dispersion problem asks for k of the points whose minimum pairwise distance r^* is as large as possible. The best known deterministic bound is $O(n^3 \log^2 n)$ (Tkachenko and Wang, SWAT 2026), obtained by binary search over a maximum-independent-set decision procedure. We make four contributions. (1) A *Delaunay certificate*: a set in convex position has all pairwise distances at least r if and only if it has a locally Delaunay triangulation whose edges all have length at least r. Its proof is short and replaces the trigonometric merging lemma of earlier work. (2) A *capped* triangulation dynamic program. Because a dispersion decision only asks whether k points fit, each chord needs to remember only $k-2$ apex angles, which we compare with exact integer in-circle tests. This gives an exact decision procedure in $O(n^2k + T_r \log k) \subseteq O(n^3 \log k)$ time and $O(n^2k)$ space, where T_r is the number of r-far triples, and an $O(n^3 \log k \log n)$ algorithm for the optimisation problem. It uses no cuttings or other geometric data structures. (3) A *search window* $g \le r^* \le \min(2g, \mathrm{per}(P)/k)$ from a farthest-first greedy set and the perimeter of the hull. (4) A study of *greedy structure*. We show that the characterisation behind the $O(n \log n)$ algorithm for $k = 3$ fails for every $k \ge 4$, and that several natural seeded greedy rules are incomplete. Each claim comes with a small explicit counterexample that is verified exhaustively. One property, that some optimal set contains a "tight" pair, survived more than 8,138 exact tests, and we state it as a conjecture. These observations give a *certified* algorithm. It runs an $O(n^2k)$ greedy and then makes one exact decision; whenever the greedy value is optimal the total time is $O(n^2k \log n + n^3 \log k)$, and the output is exact on every input. All algorithms were implemented and cross-checked against exhaustive search and an independent FMM-based solver on a dataset generated, as proposed, by taking convex hulls of random point clouds.

Contents

1 Introduction **2**

2 Preliminaries and notation **3**

3 A Delaunay certificate for r-far sets **3**

4 The capped Delaunay dynamic program **4**
4.1 Thresholds: only $k-2$ numbers per chord 5

5 Optimisation and the search window **5**

6 Greedy structure: what fails and what survives **6**
6.1 A certified algorithm 7

7 Experiments **8**

8 Discussion and open problems **9**

1 Introduction

Problem. Let P be a set of n points in the plane in *convex position*, i.e. every point of P is a vertex of the convex hull $\mathrm{conv}(P)$. For an integer $2 \le k \le n$, the *k-dispersion* problem (also called max-min dispersion or the maximally separated set problem) asks for $S \subseteq P$, $|S| = k$, maximising $\delta(S) = \min_{p \neq q \in S} |pq|$. We write $r^* = r_k^*(P)$ for the optimum. For general planar point sets the problem is NP-hard [10]. The best general exact algorithms run in $n^{O(\sqrt{k})}$ [1] or $O(n^{k-1} \log n)$ time [3]. Convex position makes the problem polynomial.

Known results for convex position. Table 1 summarises the state of the art as far as we know it (October 2026).

Table 1: Exact algorithms for k-dispersion of n points in convex position.

Reference	Time	Remarks
Singireddy, Basappa, Mitchell [7]	$O(2^k n^2 \log^2 n)$	bounded search tree, FPT in k
Singireddy, Basappa, Mitchell [7]	$O(n^4 k^2)$	first polynomial algorithm
Tkachenko, Wang [8]	$O(n^{7/2} \log n)$; $O(n^{37/11} \log n)$ exp.	cuttings; randomised
Tkachenko, Wang [9]	$O(n^3 \log^2 n)$	via $O(n^3 \log n)$ max. independent set
Kobayashi et al. [6]	$O(n^2)$	$k = 3$
Tkachenko, Wang [8]	$O(n \log^2 n)$; $O(n \log n)$ exp.	$k = 3$
This work (Thm. 11)	$O(n^3 \log k \log n)$	decision $O(n^3 \log k)$, exact integer predicates
This work (Thm. 14)	$O(n^2 k \log n + n^3 \log k)$	whenever the greedy value is optimal; always exact

Contributions.

1. **Delaunay certificate** (Theorem 3). A set S in convex position is r-far (all pairwise distances $\ge r$) iff it has a triangulation in which every diagonal is locally Delaunay and every edge has length $\ge r$. The proof combines the classical Delaunay lemma with a short inscribed-angle argument (Lemma 2).

2. **Capped Delaunay DP** (Section 4). The certificate makes the decision problem a dynamic program over triangulations with one local angle condition per diagonal. We show (Lemma 7) that the DP value of a chord, as a function of the angle constraint imposed by its parent, is a step function determined by $k - 2$ apex points. This yields an $O(n^2 k + T_r \log k)$ decision procedure using only orientation and in-circle predicates on integers.

3. **Search window** (Lemma 10): $g \le r^* \le \min\{2g, \mathrm{per}(P)/k\}$, where g is the value of the farthest-first greedy k-set and $\mathrm{per}(P)$ the hull perimeter.

4. **Greedy structure** (Section 6). The arc characterisation used for $k = 3$ by Tkachenko and Wang (their Lemma 16) fails for every $k \ge 4$; seeded alternating greedy rules are incomplete (explicit counterexamples with 6, 8 and 10 points). The tight-pair property (Conjecture 13) survived every test.

5. **Certified algorithm** (Theorem 14) and a full implementation with exhaustive cross-verification (Section 7).

Positioning. Our worst-case optimisation bound $O(n^3 \log k \log n)$ improves on $O(n^3 \log^2 n)$ [9] by a factor of $\log n / \log k$, and is better than the FPT bound $O(2^k n^2 \log^2 n)$ once $2^k \gtrsim n / \log n$. It is therefore the best known deterministic bound roughly when $\log_2 n \lesssim k \leq n^{o(1)}$. The decision bound $O(n^3 \log k)$ improves the $O(n^3 \log n)$ decision bound of [9] whenever $k = n^{o(1)}$. Our subproblems are, in substance, those of [8]; what is new is the certificate-based correctness proof, the k-capped threshold representation, the exact predicate-only implementation, the window, and the structural results. We could not access the full text of [9] and compare only with its stated bounds.

2 Preliminaries and notation

Table 2: Notation.

$P = (p_0, \ldots, p_{n-1})$	input points in strictly convex position, counterclockwise (ccw); indices mod n
$\lvert pq \rvert$, $D(i,j)$	Euclidean distance; squared distance $\lvert p_i p_j \rvert^2$ (an integer)
$P(i,j)$	the points strictly between p_i and p_j going ccw from p_i
r-far	a set whose pairwise distances are all $\geq r$
$\mathrm{far}_r(i,j)$	the predicate $\lvert p_i p_j \rvert \geq r$
$\angle abc$	the angle at b in triangle abc, in $(0, \pi)$
r^*	$\max_{\lvert S \rvert = k} \delta(S)$, the optimal value
$a(q), b(q)$	first point ccw (resp. clockwise) from q at distance $\geq r$
$F(i,j \mid \varphi)$	DP value of chord (i,j) under apex-angle bound φ (Def. 4)
$\tau_c(i,j)$	c-th threshold angle of chord (i,j) (Lemma 7); $K = k-2$
T_r	number of ccw triples (i,l,j) with $\mathrm{far}_r(i,l), \mathrm{far}_r(l,j), \mathrm{far}_r(i,j)$
$\mathrm{per}(P)$	perimeter of $\mathrm{conv}(P)$

A *triangulation* of a set S in convex position is a maximal set of pairwise non-crossing segments (edges) between points of S. Its edges are the hull sides of S and the *diagonals*. A diagonal ab with incident triangles abc and bad is *locally Delaunay* if d is not strictly inside the circumcircle of abc; since $acbd$ is a convex quadrilateral this is equivalent to $\angle acb + \angle adb \leq \pi$. A triangulation is *Delaunay* if no point of S lies strictly inside the circumcircle of any of its triangles.

Exact predicates. For integer points with coordinates of absolute value at most 2^{29}, the orientation and in-circle determinants fit in signed 128-bit integers. All angle comparisons used below reduce to these predicates via two facts. For x, y on the same side of a line ij: $\angle ixj > \angle iyj$ iff x is strictly inside the circle through i, y, j (inscribed angle theorem). For a convex quadrilateral i, x, l, j in this order: $\angle ixl + \angle ijl > \pi$ iff x is strictly inside the circle through i, l, j.

3 A Delaunay certificate for r-far sets

Lemma 1 (Delaunay lemma [4, 5, 2])**.** *If every diagonal of a triangulation T of a finite point set S is locally Delaunay, then T is a Delaunay triangulation of S.*

Lemma 2. *Let T be a Delaunay triangulation of a set S in strictly convex position, $\lvert S \rvert \geq 3$. Then $\min_{p \neq q \in S} \lvert pq \rvert$ is attained by an edge of T.*

Proof. Let p, q be a closest pair and suppose pq is not an edge of T. The triangles of T incident to p partition the interior angle of $\mathrm{conv}(S)$ at p into wedges. The direction of q from p is not along an edge, since S is strictly convex. So it lies strictly inside the wedge of some triangle puv, and because q is a vertex of $\mathrm{conv}(S)$ different from p, u, v, the segment pq crosses uv. Let C be the circumcircle of puv and A the arc of C from u to v that does not contain p. Every chord from p to a point of A lies in the wedge, and q is not strictly inside C because T is Delaunay.

Hence the segment pq leaves the closed disk bounded by C through a point $x \in A$, and $|pq| \geq |px|$. For x moving along A from u to v, the distance $|px| = 2R\sin(\theta/2)$ is a unimodal function of the central angle θ between p and x (which first increases and then decreases), so it is minimised at an endpoint: $|px| \geq \min\{|pu|, |pv|\}$. Hence some edge, pu or pv, has length at most $|pq|$, the minimum distance. □

Theorem 3 (Delaunay certificate)**.** *Let S be in strictly convex position, $|S| \geq 3$, and $r > 0$. Then S is r-far if and only if S has a triangulation T such that (a) every edge of T has length $\geq r$ and (b) every diagonal of T is locally Delaunay.*

Proof. ($\Leftarrow$) By Lemma 1, T is Delaunay; by Lemma 2 the minimum pairwise distance equals the minimum edge length of T, which is $\geq r$. ($\Rightarrow$) A Delaunay triangulation of S exists (triangulate the cells of the Delaunay subdivision) and satisfies (b). All its edges join points of S, so (a) holds because S is r-far. □

Conditions (a) and (b) are *local*: (a) concerns single edges and (b) single pairs of adjacent triangles. This is what makes a dynamic program possible. The same structure underlies the algorithm of Tkachenko and Wang [8], whose correctness proof instead uses a seven-term distance inequality (their Lemma 14); Theorem 3 gives a shorter route.

4 The capped Delaunay dynamic program

Fix r and write far for far_r. A chord (i, j) is *admissible* if $\mathrm{far}(i, j)$; a point $l \in P(i, j)$ is an *admissible apex* of (i, j) if $\mathrm{far}(i, l)$ and $\mathrm{far}(l, j)$.

Definition 4. For an admissible chord (i, j), an (i, j)-*configuration* is a pair (X, T) with $X \subseteq P(i, j)$ and T a triangulation of $X \cup \{p_i, p_j\}$ such that (a) all edges of T have length $\geq r$ and (b) all diagonals of T are locally Delaunay. If $X \neq \emptyset$, the *apex* of (X, T) is the third vertex l of the triangle of T on p_ip_j. For $\varphi \in (0, \pi]$ let $F(i, j \mid \varphi)$ be the maximum $|X|$ over configurations with $X = \emptyset$ or $\angle p_ilp_j \leq \varphi$.

The bound φ encodes the parent: if p_ip_j becomes a diagonal whose other triangle is $p_jp_ip_t$, local Delaunayhood of p_ip_j is exactly $\angle p_ilp_j \leq \pi - \angle p_ip_tp_j$.

Lemma 5 (recurrence)**.** *For every admissible chord (i, j) and $\varphi \in (0, \pi]$,*

$$F(i, j \mid \varphi) = \max\Big(\{0\} \cup \{\, 1 + F(i, l \mid \pi - \angle p_ip_jp_l) + F(l, j \mid \pi - \angle p_lp_ip_j) \\ : \ l \text{ admissible apex}, \ \angle p_ilp_j \leq \varphi\}\Big).$$

Proof. ($\geq$) Given l and optimal configurations (X_1, T_1) for (i, l) and (X_2, T_2) for (l, j) under the stated bounds, let $X = X_1 \cup X_2 \cup \{l\}$ and $T = T_1 \cup T_2 \cup \{p_ip_lp_j\}$. The new edges p_ip_l, p_lp_j, p_ip_j have length $\geq r$. If $X_1 \neq \emptyset$, p_ip_l is a diagonal of T with triangles $p_ip_lp_j$ and the apex triangle of T_1, and the bound $\pi - \angle p_ip_jp_l$ is exactly its local Delaunay condition; symmetrically for p_lp_j. All other diagonals are diagonals of T_1 or T_2. The apex of (X, T) is l, with $\angle p_ilp_j \leq \varphi$. ($\leq$) If an optimal configuration has $X \neq \emptyset$, let l be its apex. Removing triangle $p_ip_lp_j$ splits T into configurations for (i, l) and (l, j) (both chords have length $\geq r$ by (a)). By (b), their apex angles obey the stated bounds. Hence $|X| \leq 1 + F(i, l \mid \cdot) + F(l, j \mid \cdot)$. □

Theorem 6 (decision)**.** *For $k \geq 3$: P has an r-far k-set iff some admissible chord (i, j) has $F(i, j \mid \pi) \geq k - 2$.*

Proof. ($\Rightarrow$) Let $S = \{q_1, \ldots, q_k\}$ be r-far in ccw order and T a Delaunay triangulation of S. With $p_i = q_1$, $p_j = q_k$ and $X = S \setminus \{q_1, q_k\} \subseteq P(i, j)$, (X, T) is an (i, j)-configuration by Theorem 3($\Rightarrow$). Its apex angle is $< \pi$, so $F(i, j \mid \pi) \geq k-2$. ($\Leftarrow$) A configuration with $|X| \geq k-2$ is a triangulation of $X \cup \{p_i, p_j\}$ satisfying (a) and (b). By Theorem 3 this set is r-far; any k of its points form the required set. □

Algorithm 1 CappedDecision(P, k, r)

for $g = 2, \dots, n-1$ **do** ▷ chords by increasing ccw gap
  for each admissible chord (i, j) with $j = i + g$ **do**
    $best[1..K] \leftarrow \perp$
    for each admissible apex $l \in P(i, j)$ **do**
      $q_1 \leftarrow \#\{c : L_c(i, l)$ not strictly inside $\bigcirc(p_i, p_l, p_j)\}$ ▷ binary search
      $q_2 \leftarrow \#\{c : L_c(l, j)$ not strictly inside $\bigcirc(p_i, p_l, p_j)\}$
      $v \leftarrow \min(K, 1 + q_1 + q_2)$; keep in $best[v]$ the apex of smaller angle
    end for
    $L_c(i, j) \leftarrow$ smallest-angle apex among $best[c..K]$ for $c = K, \dots, 1$ (suffix minima)
  end for
end for
return true iff some admissible chord has $L_K(i, j) \neq \perp$

4.1 Thresholds: only $k-2$ numbers per chord

Fix an admissible chord (i, j). For an admissible apex l let $v(l) = 1 + F(i, l \mid \pi - \angle p_i p_j p_l) + F(l, j \mid \pi - \angle p_l p_i p_j)$ and, for $c \geq 1$, $\tau_c(i, j) = \min\{\angle p_i l p_j : l$ admissible, $v(l) \geq c\}$ (and $+\infty$ if no such l exists).

Lemma 7. *$\tau_1(i, j) \leq \tau_2(i, j) \leq \cdots$ and $F(i, j \mid \varphi) = \max\{c \geq 0 : \tau_c(i, j) \leq \varphi\}$ (with $\tau_0 = 0$). Moreover, with $K = k-2$ and $F_K = \min(F, K)$, the values $\tau_1, \dots, \tau_K$ of every chord are determined by the values F_K of shorter chords.*

Proof. Monotonicity holds because the sets $\{l : v(l) \geq c\}$ shrink as c grows. By Lemma 5, $F(i, j \mid \varphi) \geq c$ iff some admissible l with $\angle p_i l p_j \leq \varphi$ has $v(l) \geq c$, iff $\tau_c \leq \varphi$. For the last claim, $\min(K, v(l)) = \min(K, 1 + F_K(i, l \mid \cdot) + F_K(l, j \mid \cdot))$, and τ_c for $c \leq K$ depends on v only through $\min(K, v)$. □

We store $\tau_c(i, j)$ as the apex point $L_c(i, j)$ that attains it. The query $F_K(i, l \mid \pi - \angle p_i p_j p_l) \geq c$ is then "$L_c(i, l)$ is not strictly inside the circle through p_i, p_l, p_j", which is monotone in c, so the count is found by binary search with $O(\log K)$ in-circle tests.

Theorem 8. *Algorithm 1 decides whether P has an r-far k-set in $O(n^2 k + T_r \log k) \subseteq O(n^3 \log k)$ time and $O(n^2 k)$ space, using only exact integer predicates. A witness is reconstructed in $O(k \log k)$ further time.*

Proof. Correctness follows from Theorem 6 and Lemma 7; chords are processed after all shorter chords. The loop body runs once per admissible triple, T_r times, at $O(\log K)$ cost. Initialising and finishing each chord costs $O(K)$. For the witness, descend from a chord with $L_K \neq \perp$: at chord (i, j) with demand c, take $l = L_c(i, j)$, send $c_1 = \min(q_1, c-1)$ to (i, l) and $c - 1 - c_1 \leq q_2$ to (l, j). □

Remark 9. Without capping, a chord's step function can have $\Theta(n)$ steps. Storing all of them gives $O(n^3 \log n)$, which matches the decision bound of [9] for unit disks. Capping is what reduces the logarithm to $\log k$, the memory to $O(n^2 k)$, and the per-chord work to one linear pass.

5 Optimisation and the search window

Lemma 10 (search window)**.** *Let G be the farthest-first greedy k-set (start anywhere; repeatedly add the point farthest from the chosen ones) and $g = \delta(G)$. Then $g \leq r^* \leq \min\{2g,\ \mathrm{per}(P)/k\}$.*

Proof. $g \leq r^*$ because G is a feasible k-set. Let G' be the first $k-1$ greedy points. The distances at which points are added are non-increasing, so every point of P lies within g of G'. Among the

k points of an optimal set, two share a nearest point of G', so $r^* \le 2g$. Finally let S^* be optimal. For $k \ge 3$, $\mathrm{conv}(S^*)$ is a convex k-gon whose sides all have length $\ge r^*$, so its perimeter is at least kr^*. Since $\mathrm{conv}(S^*) \subseteq \mathrm{conv}(P)$ and perimeter is monotone under inclusion of convex sets, $kr^* \le \mathrm{per}(P)$. For $k = 2$, r^* is the diameter, which is at most $\mathrm{per}(P)/2$. □

Theorem 11. *k-dispersion for n points in convex position with integer coordinates in $[-2^{29}, 2^{29}]$ can be solved exactly in $O(n^3 \log k \log n)$ time and $O(n^2 k)$ space.*

Proof. r^* is a pairwise distance, so we binary search the sorted distinct squared distances inside the window of Lemma 10 ($O(n^2 \log n)$ to sort), with Algorithm 1 as the decision procedure ($O(\log n)$ calls). The witness at r^* is an optimal set. □

6 Greedy structure: what fails and what survives

For $k = 3$ there is an $O(n \log n)$ decision procedure [8], based on the following characterisation (their Lemma 16, from [6]). Let $A(i) = P[a(p_i), b(p_i)]$ be the ccw arc between the first far points on either side of p_i.

$$(G_k) \qquad P \text{ has an } r\text{-far } k\text{-set} \iff \exists i : \ A(i) \text{ contains an } r\text{-far } (k-1)\text{-set}.$$

G_3 is true. A fast algorithm for $k \ge 4$ would follow from G_k or from a greedy that grows a solution outward from a seed. We tested these properties exhaustively on random convex sets (all arithmetic exact). In what follows, a *seeded alternating greedy* keeps a ccw and a clockwise frontier and alternately adds the first point beyond a frontier that is r-far from every chosen point. Both starting sides are tried, and a step that finds no point fails the run.

C_1: some r-far k-set contains a pair $q, a(q)$ (a *tight pair*);

C_1': some r-far k-set contains q, $a(q)$ and $b(q)$;

V_1: the alternating greedy seeded with $\{q, a(q), b(q)\}$ succeeds for some q; V_3: as V_1 but with every left/right pattern;

V_5: the alternating greedy seeded with $\{q, a(q)\}$ succeeds for some q; V_4: as V_5 but with every pattern.

Proposition 12. *G_4 is false; C_1', V_1 and V_3 are false for $k = 4$; V_5 is false for $k = 6$. Table 3 lists explicit counterexamples (integer coordinates, ccw order), verified by exhaustive search in exact arithmetic (`convex/experiments/check_counterexamples.py`). The random tests of Table 4 also found failures of G_5 and G_6.*

Table 3: Minimal counterexamples found (points given in ccw order; indices are 0-based in that order). For G_4: no r-far 4-set exists, yet an arc $A(i)$ contains an r-far triple. For V_1/V_5: an r-far k-set exists (shown) but the greedy fails from every seed.

Property	n	k	r^2	an r-far k-set
G4	6	4	320756	none
`(-976,42), (4,-49), (971,-41), (979,4), (964,29), (398,49)`				
V1	8	4	2556513005	0, 3, 4, 7
`(-90482,144), (-45601,-629), (13859,-779), (38501,-677), (97951,32), (57181,535), (4941,792), (-19180,765)`				
V5	10	6	878078401	0, 1, 2, 5, 6, 7
`(-94942,276), (-57395,-1876), (-27771,-2581), (-2908,-2795), (62044,-1720), (87508,-655), (48547,2138), (2056,2796), (-24473,2629), (-70081,1423)`				

All counterexamples are thin, "two-chain" convex polygons. In such polygons the cyclic order is far from geometric proximity: points adjacent in space lie on opposite chains and far apart in the cyclic order. Greedy growth from one seed commits early to points on one chain that later block points on the other.

Table 4: Exhaustive tests of the properties (counts of (P, k, r) tested; failures). The C_1 row comes from the exact DP-based check (`v5check -c1`) on larger instances.

Property	k	tests	failures
G_k (Lemma 16 generalised)	3	16026	0
G_k (Lemma 16 generalised)	4	14618	245
G_k (Lemma 16 generalised)	5	14865	228
G_k (Lemma 16 generalised)	6	14002	121
C_1 (a tight pair $q, a(q)$)	3	8835	0
C_1 (a tight pair $q, a(q)$)	4	5293	0
C_1 (a tight pair $q, a(q)$)	5	3677	0
C_1 (a tight pair $q, a(q)$)	6	2472	0
C_1' ($q, a(q), b(q)$ together)	3	8835	0
C_1' ($q, a(q), b(q)$ together)	4	5293	0
C_1' ($q, a(q), b(q)$ together)	5	3677	0
C_1' ($q, a(q), b(q)$ together)	6	2472	0
V_5 (hunt, $n \le 22$, $k \le 9$)	3–9	201,422	5
C_1 (hunt, $n \le 30$, $k \le 12$)	3–12	8,138	0

Conjecture 13 (tight pair)**.** *If P (in convex position) has an r-far k-set, then it has one containing a pair $q, a(q)$.*

Conjecture 13 would let the top level of the DP use only the n chords $(a(q), q)$. More importantly, it is the first step of any seed-and-grow algorithm. Property V_4 (all patterns from a tight pair) also had no failures in our tests; if true, it yields an $O(2^k n^2 k)$ algorithm. The counterexamples show that a correct proof of such a property must use more than alternation or a three-point seed.

6.1 A certified algorithm

Seeded greedies are *sound*: they only ever output r-far sets. This gives an algorithm that is exact on every input and fast whenever a greedy is optimal.

Algorithm 2 CERTIFIED(P, k)

```
W ← sorted candidate distances in the window of Lemma 10
ρ ← largest element of W found by binary search with the seeded greedy (seeds {q, a(q), b(q)}
    and {q, a(q)}, both alternation orders) as predicate
if ρ is the last element of W or CappedDecision at the successor of ρ is false then
    return ρ                                                        ▷ certified optimal
end if
continue the binary search above ρ with CappedDecision
```

Theorem 14. *Algorithm 2 always returns r^*. The greedy predicate costs $O(n^2 k)$ per call. Hence the total time is $O(n^2 k \log n + n^3 \log k)$ whenever the greedy binary search returns r^*, and $O(n^3 \log k \log n)$ in the worst case.*

Proof. Every greedy success exhibits an r-far set, so $\rho \le r^*$. If the exact decision at the successor ρ^+ of ρ fails, then $r^* < \rho^+$ by monotonicity, so $r^* = \rho$. Otherwise the exact binary search runs on $[\rho^+, \max W]$, which contains r^*. Per seed, each frontier pointer only moves forward (a rejected point stays rejected because the chosen set only grows), so a greedy run costs $O(nk)$, and there are $O(n)$ seeds. □

7 Experiments

Dataset (random points → convex hull). `convex/gen_convex_dataset.py` samples N random integer points (coordinates up to 10^6) from a distribution, computes the strictly convex hull, and keeps n of its vertices (a subset of a convex-position set is in convex position). Distributions: uniform square and Gaussian (small hulls, $n \le 30$ resp. 14), uniform disk, thin annulus, thin 1:20 ellipse, points on a circle, and *cocircular* points (all 324 lattice points on $x^2 + y^2 = 32045^2$, the worst case for in-circle tests). There are three splits: *verify* ($n \le 14$, 280 instances), *medium* ($n \le 40$, 68) and *scaling* ($n \le 200$, 93).

Correctness. Each instance was solved by the capped DP, the certified algorithm, the Tkachenko–Wang DP (direct $O(n^4)$-per-decision implementation), exhaustive search and, on *verify*, the general FMM-based solver of this project. Every returned set was re-checked independently in Python.

Table 5: Optimum agrees with exhaustive search and witness valid (per distribution). Verify split: 280/280 instances fully correct; medium: 68/68.

verify ($n \le 14$)					
Distribution	inst.	capped	certified	TW	FMM
`square`	40	40/40	40/40	40/40	40/40
`disk`	40	40/40	40/40	40/40	40/40
`gauss`	40	40/40	40/40	40/40	40/40
`annulus`	40	40/40	40/40	40/40	40/40
`thin`	40	40/40	40/40	40/40	40/40
`circle`	40	40/40	40/40	40/40	40/40
`cocircular`	40	40/40	40/40	40/40	40/40
medium ($20 \le n \le 40$)					
Distribution	inst.	capped	certified	TW	
`square`	8	8/8	8/8	8/8	
`disk`	12	12/12	12/12	12/12	
`annulus`	12	12/12	12/12	12/12	
`thin`	12	12/12	12/12	12/12	
`circle`	12	12/12	12/12	12/12	
`cocircular`	12	12/12	12/12	12/12	

Running time. Table 6 and Figure 1 report median in-process solve times (one core). The window of Lemma 10 kept a median of 6.3% (max 27.4%) of the candidate distances, saving 4.5 exact decisions on average. The certified algorithm was certified by the greedy on 93/93 scaling instances.

Table 6: Median solve time [ms] and number of exact decisions on the scaling split. TW was run only for $n \le 100$.

k	n	TW	capped	certified	TW/capped	dec. (no window)	dec. (window)	dec. (cert.)
4	25	0.4	0.2	0.2	1.5×	10	6	2
4	50	3.4	1.9	0.8	1.7×	11	8	2
4	75	18.9	6.0	2.6	3.2×	12	9	2
4	100	36.6	16.6	4.5	2.1×	13	10	2
4	150	–	52.7	18.0	–	14	11	2
4	200	–	122.9	35.1	–	15	11	2
8	25	1.9	1.7	0.7	1.2×	9	5	2
8	50	34.0	16.9	5.7	1.9×	11	7	2
8	75	153.9	53.6	17.1	3.0×	12	7	2
8	100	578.0	152.3	42.7	3.8×	14	8	2
8	150	–	460.4	135.8	–	15	8	2
8	200	–	1203.9	310.0	–	15	9	2
16	25	4.3	4.7	2.0	1.0×	9	5	2
16	50	94.1	37.9	12.8	1.7×	11	6	2
16	75	493.4	147.3	47.8	2.6×	12	7	2
16	100	1300.8	366.7	107.9	3.4×	13	7	2
16	150	–	1587.0	384.9	–	14	8	2
16	200	–	4434.8	1139.0	–	15	10	2

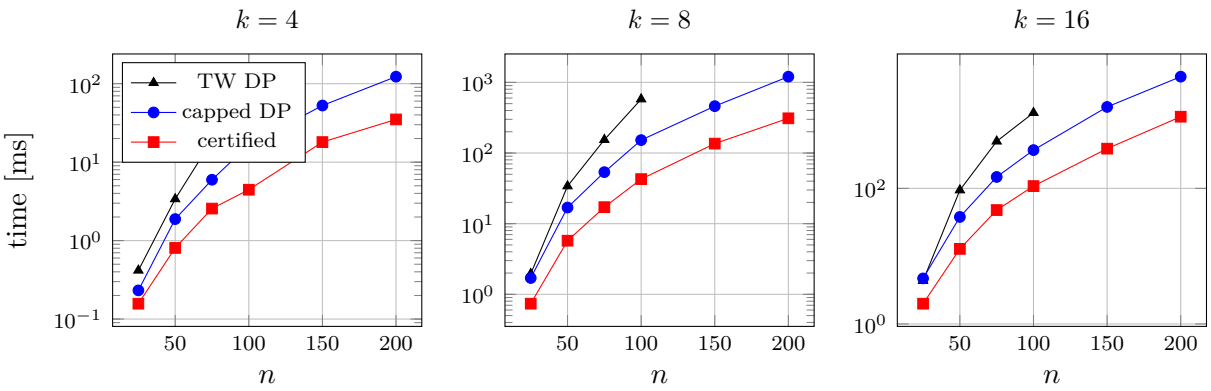


Figure 1: Median solve time vs. n over all distributions (log scale).

8 Discussion and open problems

1. Prove or refute Conjecture 13 and property V_4. A proof of V_4 would give an $O(2^k n^2 k)$ algorithm with a self-contained proof; our counterexamples rule out the simpler alternation-based variants.

2. Is there an $o(n^3)$ algorithm for fixed $k \ge 4$, matching the near-linear algorithm for $k = 3$? G_k fails, so a new characterisation is needed.

3. Can the $\log n$ factor of the binary search be removed, e.g. by parametric search over the threshold representation?

Reproducibility. All code, data and scripts are in `convex/`: `gen_convex_dataset.py`, `run_convex_experiments.py`, `experiments/*.py`, `experiments/v5check.cpp`.

References

[1] T. Akagi et al., Exact algorithms for the max-min dispersion problem, *FAW 2018*, LNCS 10823, 263–272.

[2] M. de Berg, O. Cheong, M. van Kreveld, M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed., Springer, 2008, Ch. 9.

[3] K. Chen, A. Dumitrescu, A couple of simple algorithms for k-dispersion, arXiv:2511.00692, 2025.

[4] B. Delaunay, Sur la sphère vide, *Bull. Acad. Sci. URSS* 6 (1934), 793–800.

[5] H. Edelsbrunner, *Geometry and Topology for Mesh Generation*, Cambridge University Press, 2001.

[6] Y. Kobayashi, S.-I. Nakano, K. Uchizawa, T. Uno, Y. Yamaguchi, K. Yamanaka, An $O(n^2)$-time algorithm for computing a max-min 3-dispersion on a point set in convex position, *IEICE Trans. Inf. & Syst.* E105-D(3), 2022.

[7] V. R. Singireddy, M. Basappa, J. S. B. Mitchell, Algorithms for k-dispersion for points in convex position in the plane, *Discrete Applied Mathematics*, 2026 (conference version CALDAM 2023).

[8] A. Tkachenko, H. Wang, Dominating set, independent set, discrete k-center, dispersion, and related problems for planar points in convex position, *STACS 2025*, LIPIcs 327, 73; arXiv:2501.00207.

[9] A. Tkachenko, H. Wang, Maximum independent sets in disk graphs with disks in convex position, *SWAT 2026*, LIPIcs 370, 40; arXiv:2604.10828.

[10] D.-W. Wang, Y.-S. Kuo, A study on two geometric location problems, *Inform. Process. Lett.* 28(6) (1988), 281–286.